

Agentix: An Efficient Serving Engine for LLM Agents as General Programs

Michael Luo¹, Xiaoxiang Shi², Colin Cai¹, Tianjun Zhang¹, Justin Wong¹, Yichuan Wang¹
Chi Wang³, Yanping Huang³, Zhifeng Chen³, Joseph Gonzalez¹, Ion Stoica¹

¹UC Berkeley, ²Shanghai Jiaotong University, ³Google DeepMind

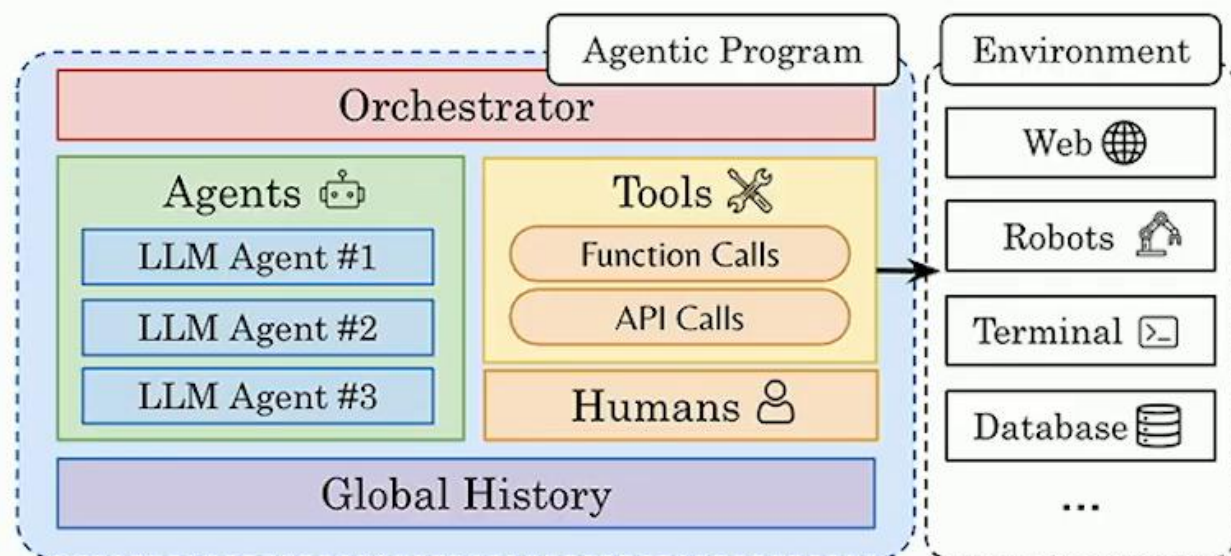


Berkeley
UNIVERSITY OF CALIFORNIA

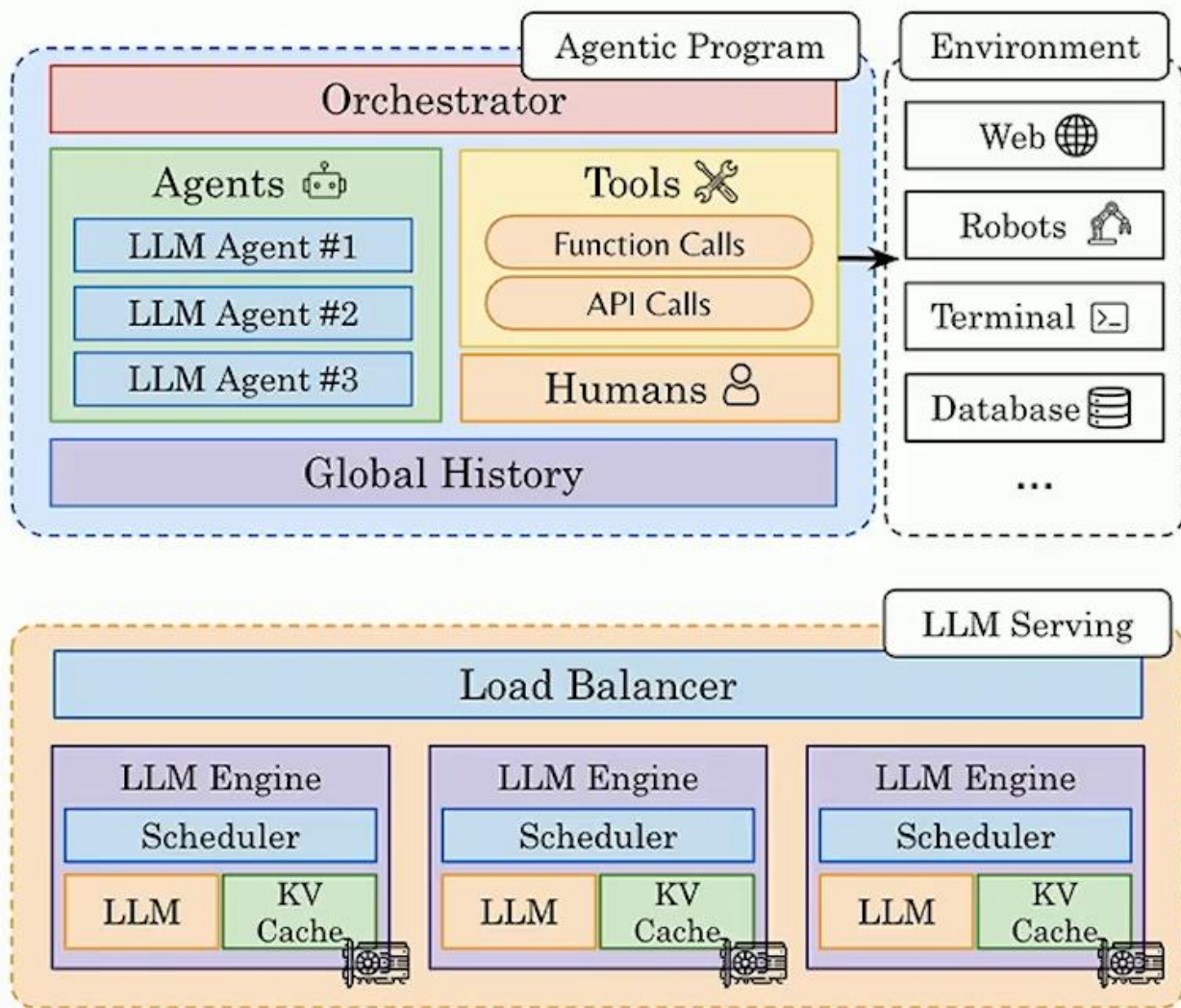


Google DeepMind

AI Ecosystem



AI Ecosystem



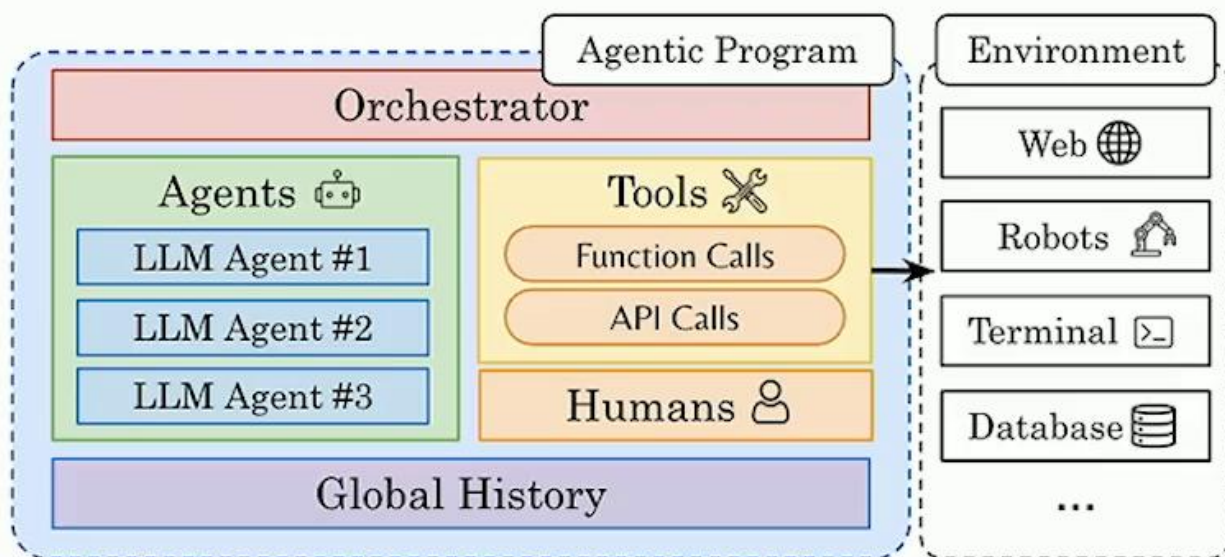
Agent Applications



LLM Serving Layer



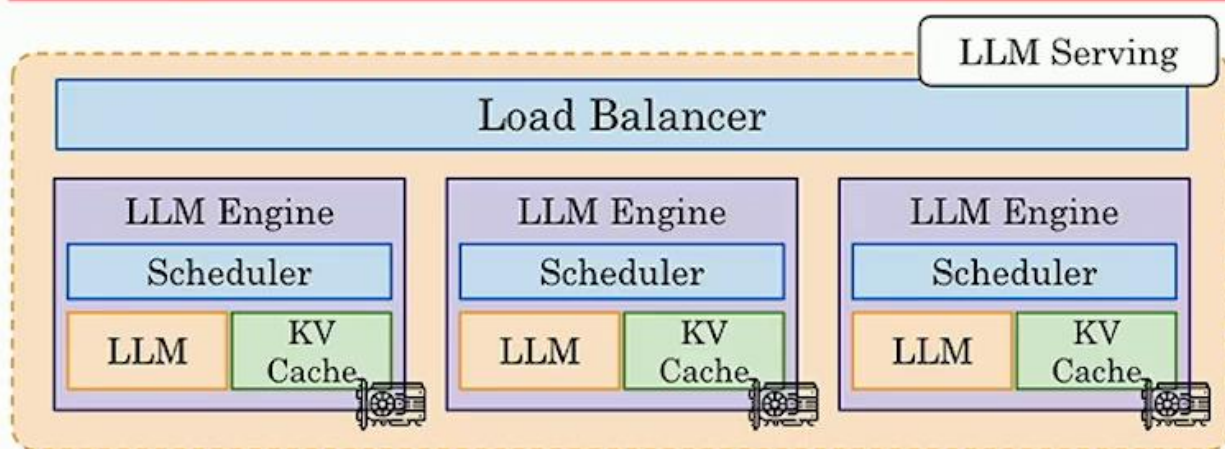
AI Ecosystem



Agent Applications



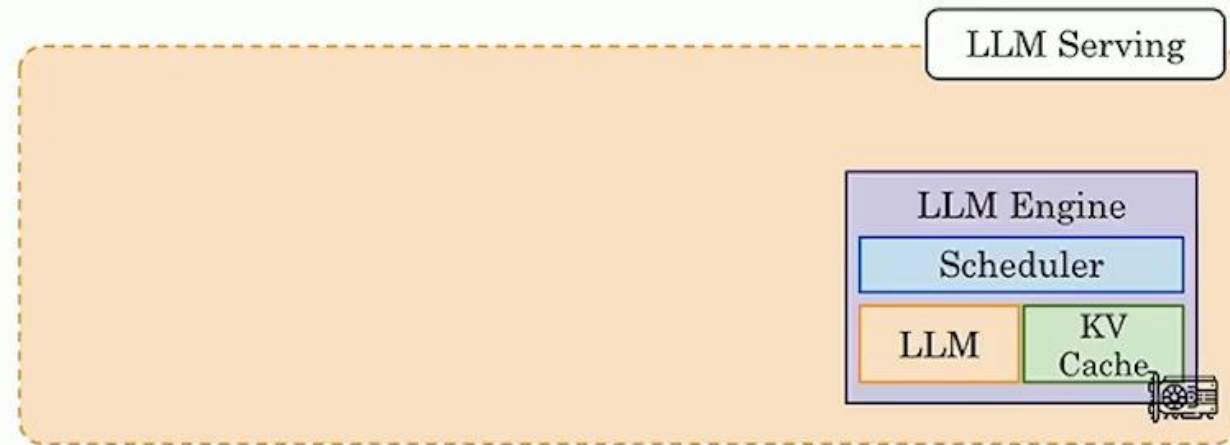
Decoupled!



LLM Serving Layer

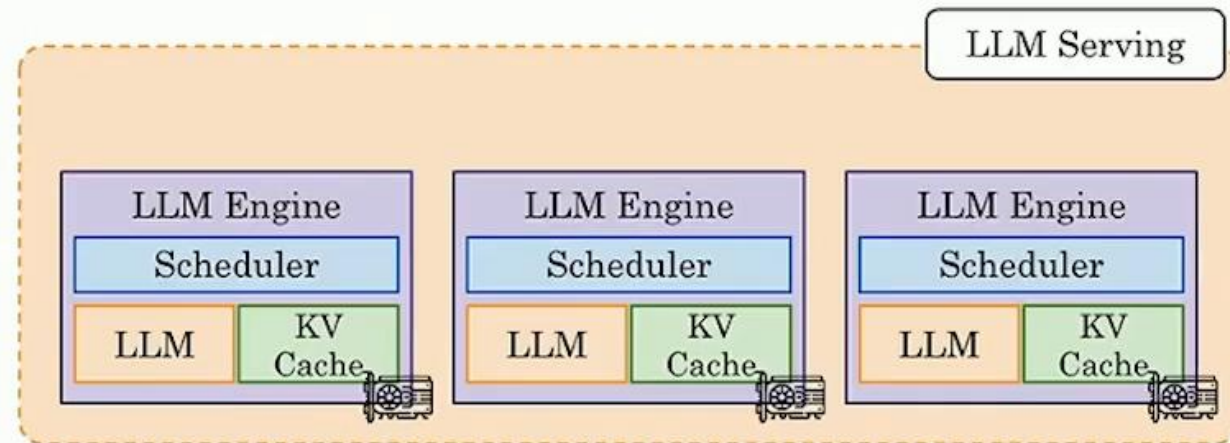


Today's AI Stack



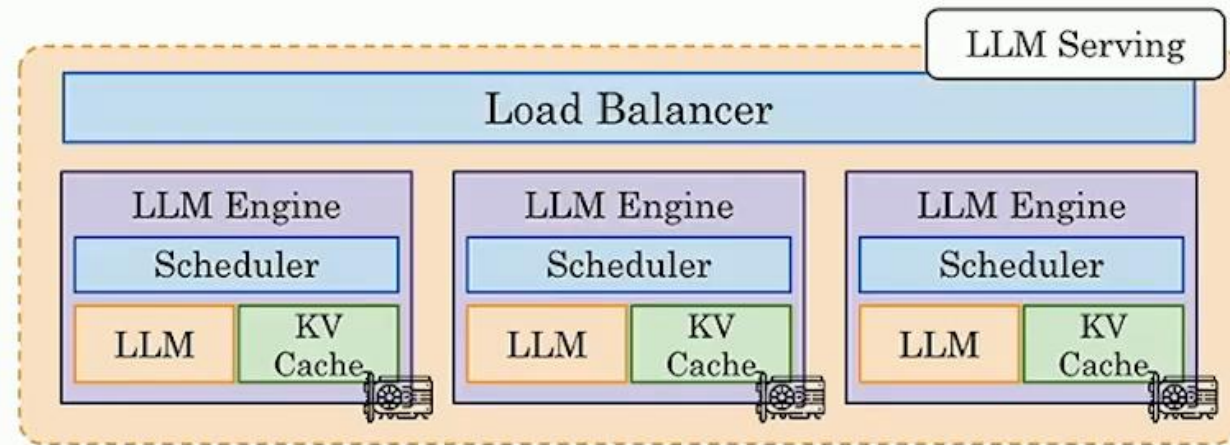
LLM Serving Layer

Today's AI Stack



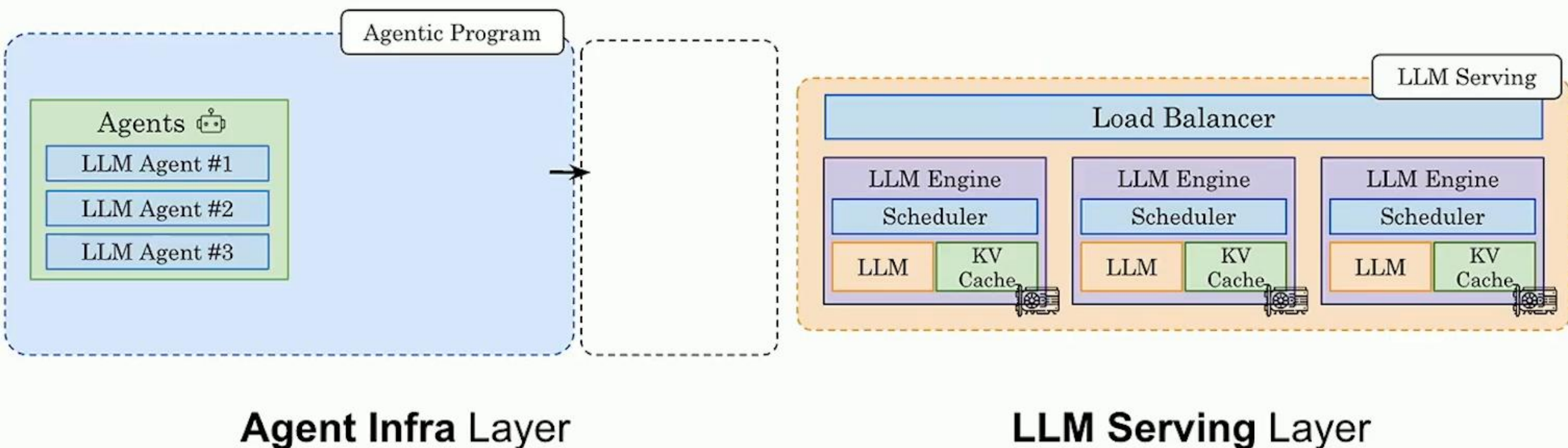
LLM Serving Layer

Today's AI Stack

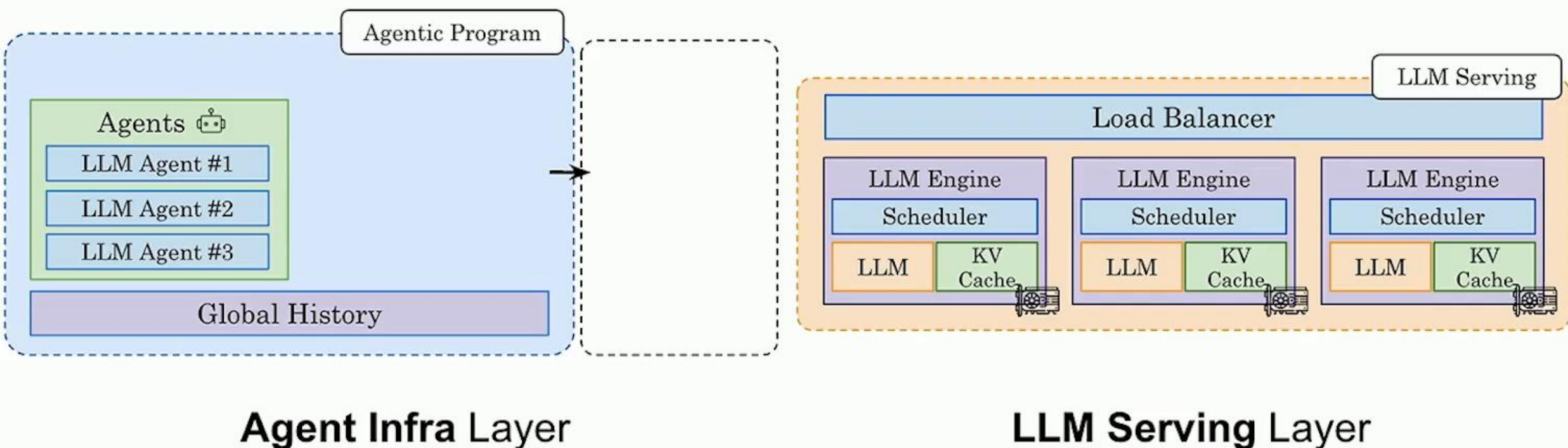


LLM Serving Layer

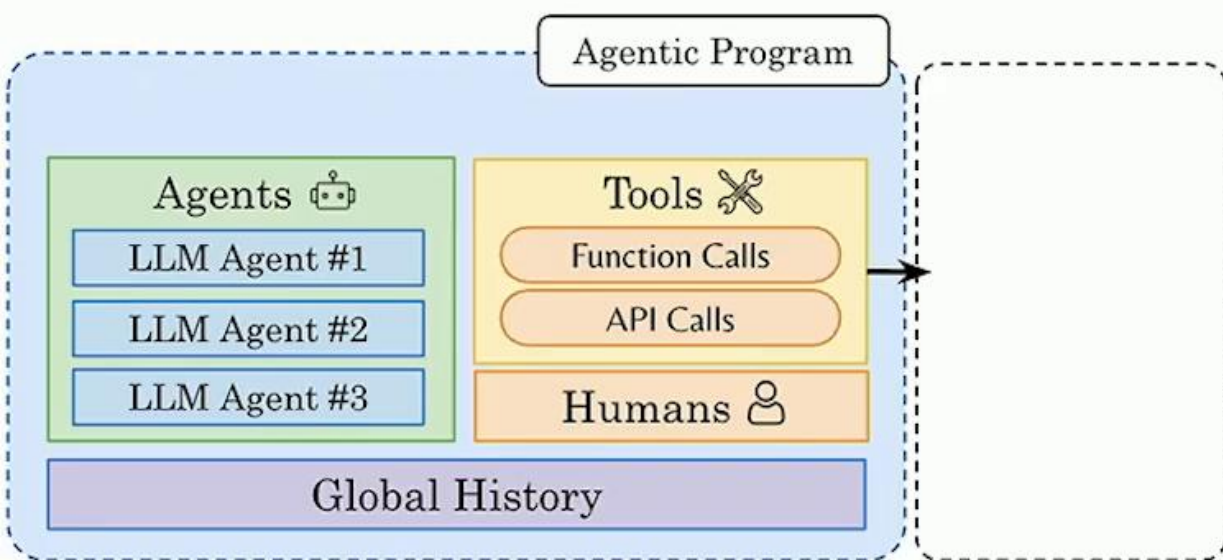
Today's AI Stack - LLM Agents



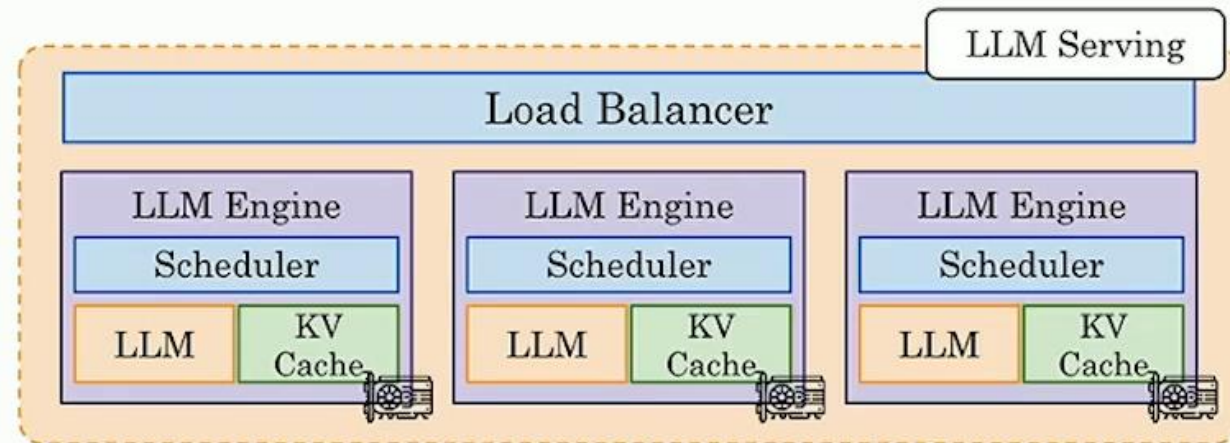
Today's AI Stack - Global Memory



Today's AI Stack - Tool/Humans

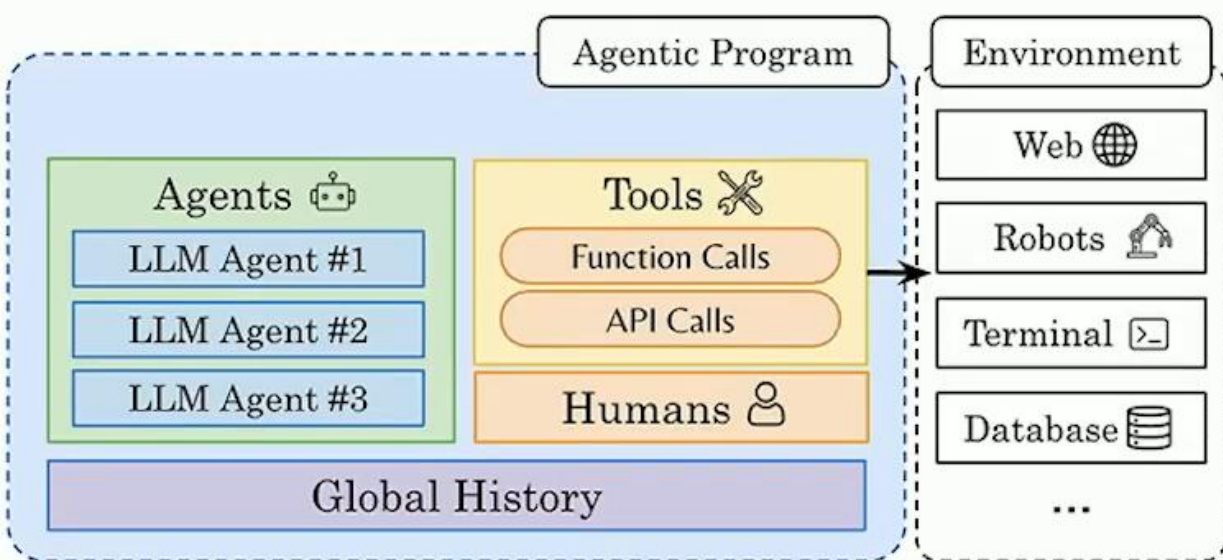


Agent Infra Layer

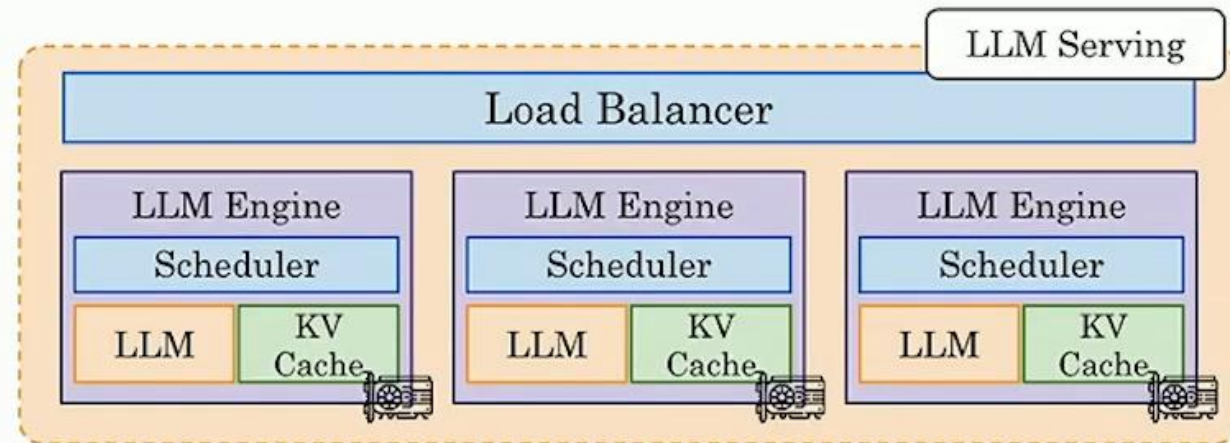


LLM Serving Layer

Today's AI Stack - External Environment



AI Application Layer

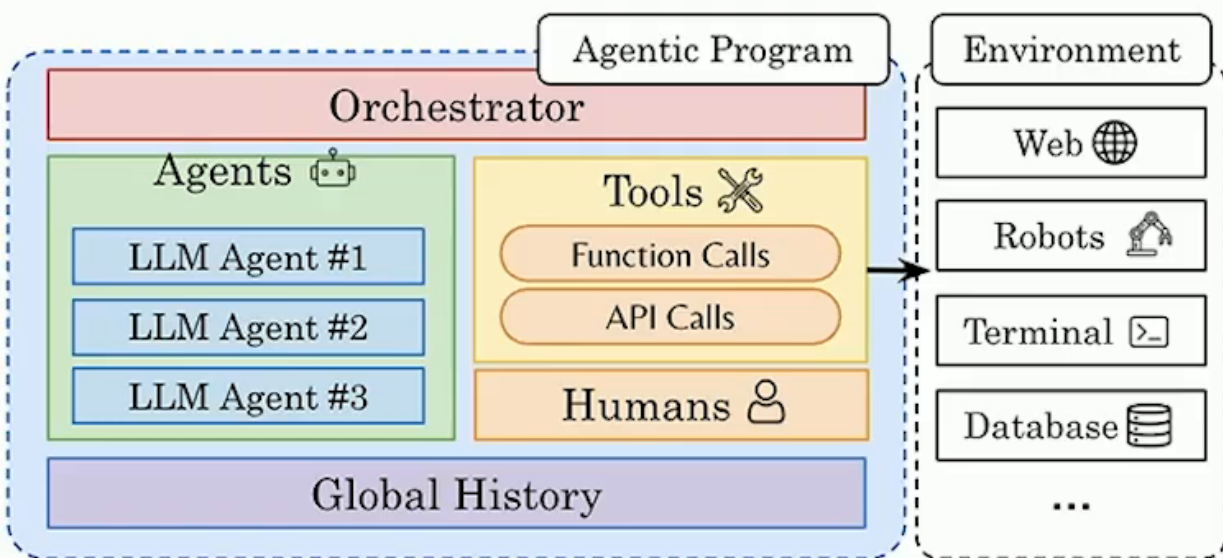
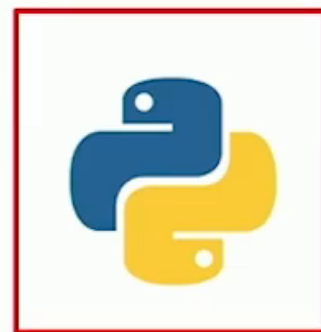


LLM Serving Layer

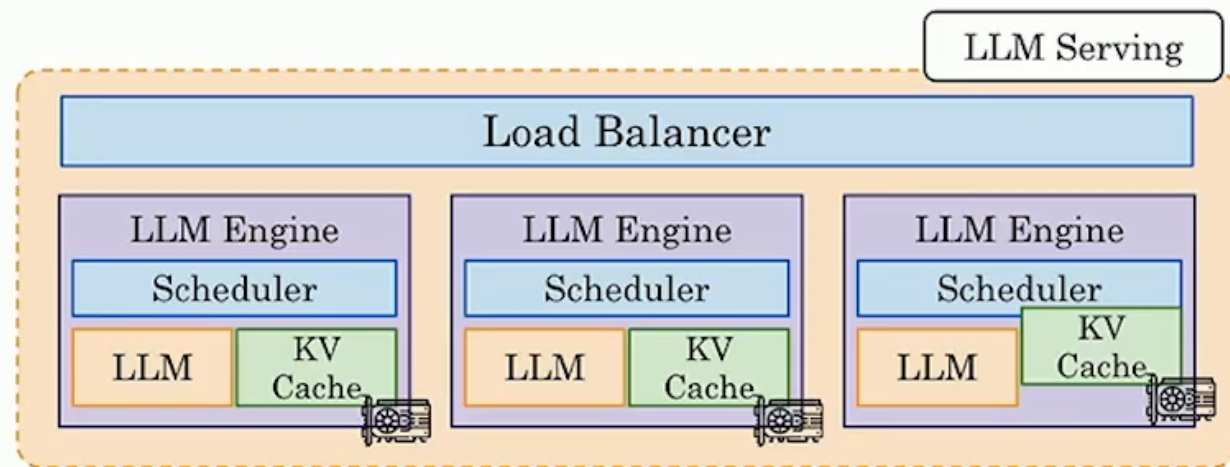
Today's AI Stack - Orchestrator



LangGraph



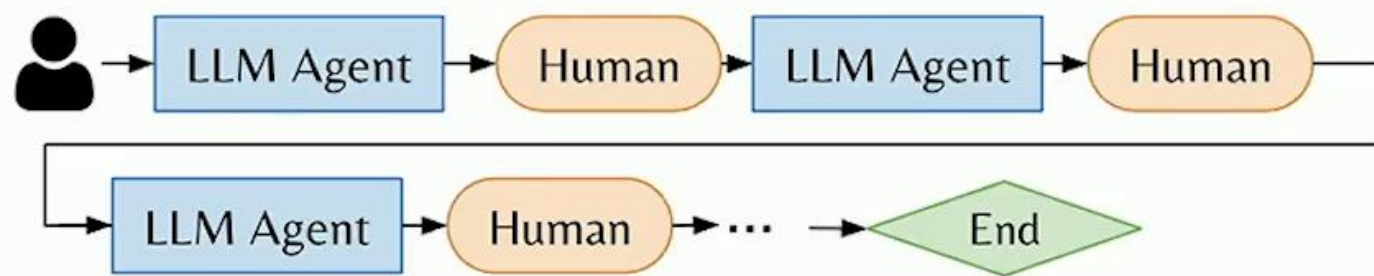
Agent Infra Layer



LLM Serving Layer

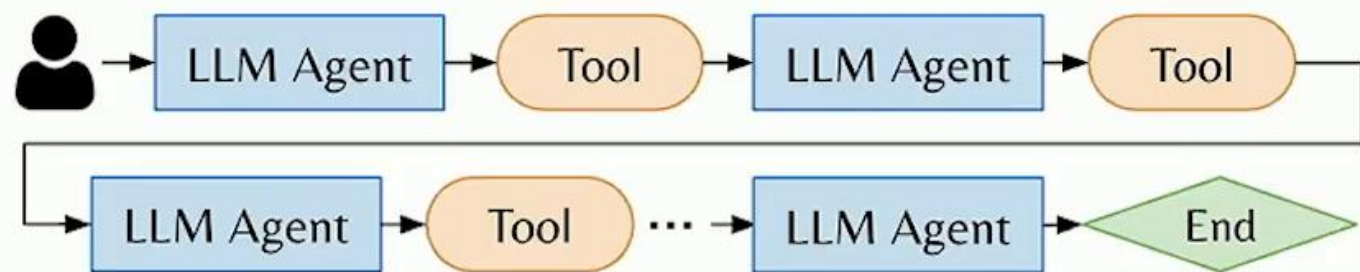
Complication: Agents are increasingly complex!

Illustration of Execution DAG



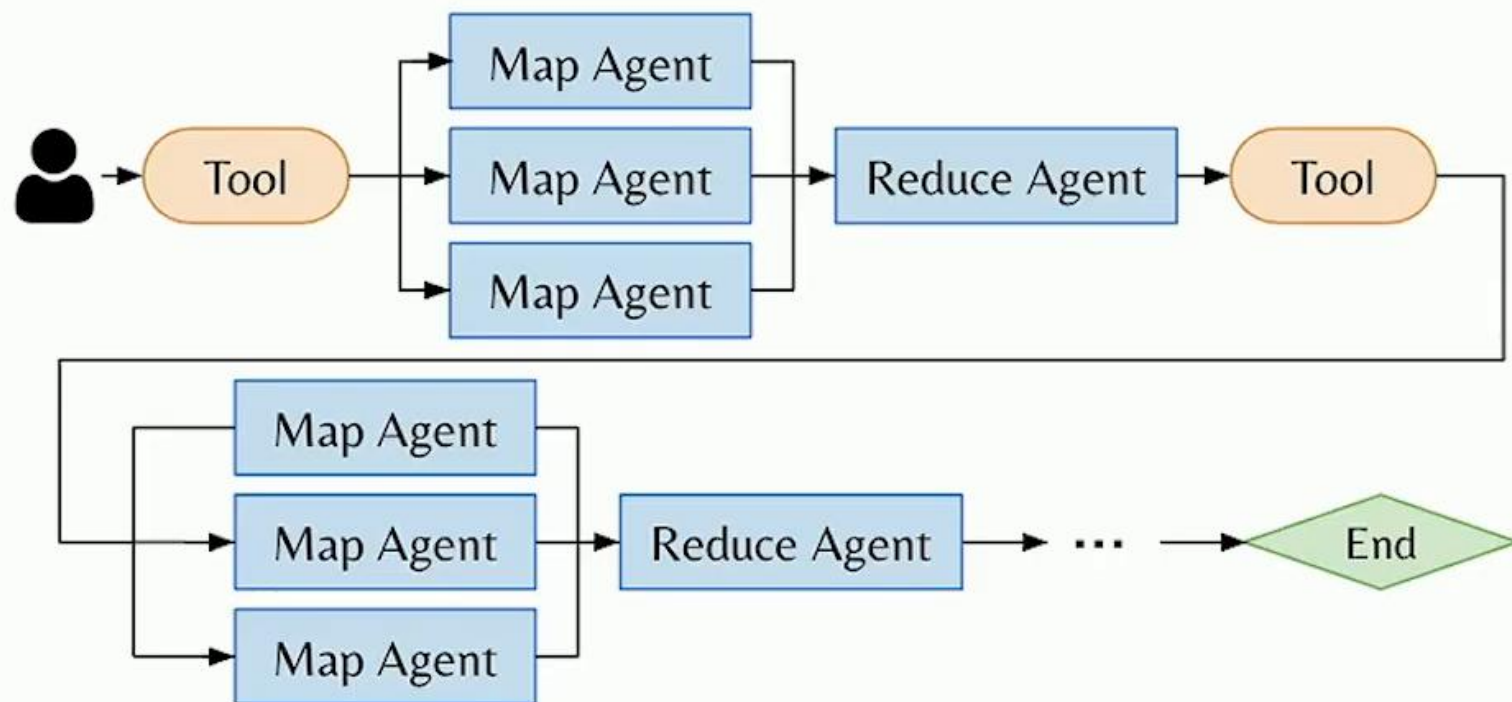
Chatbot (2022-23)

AI Applications are complex!



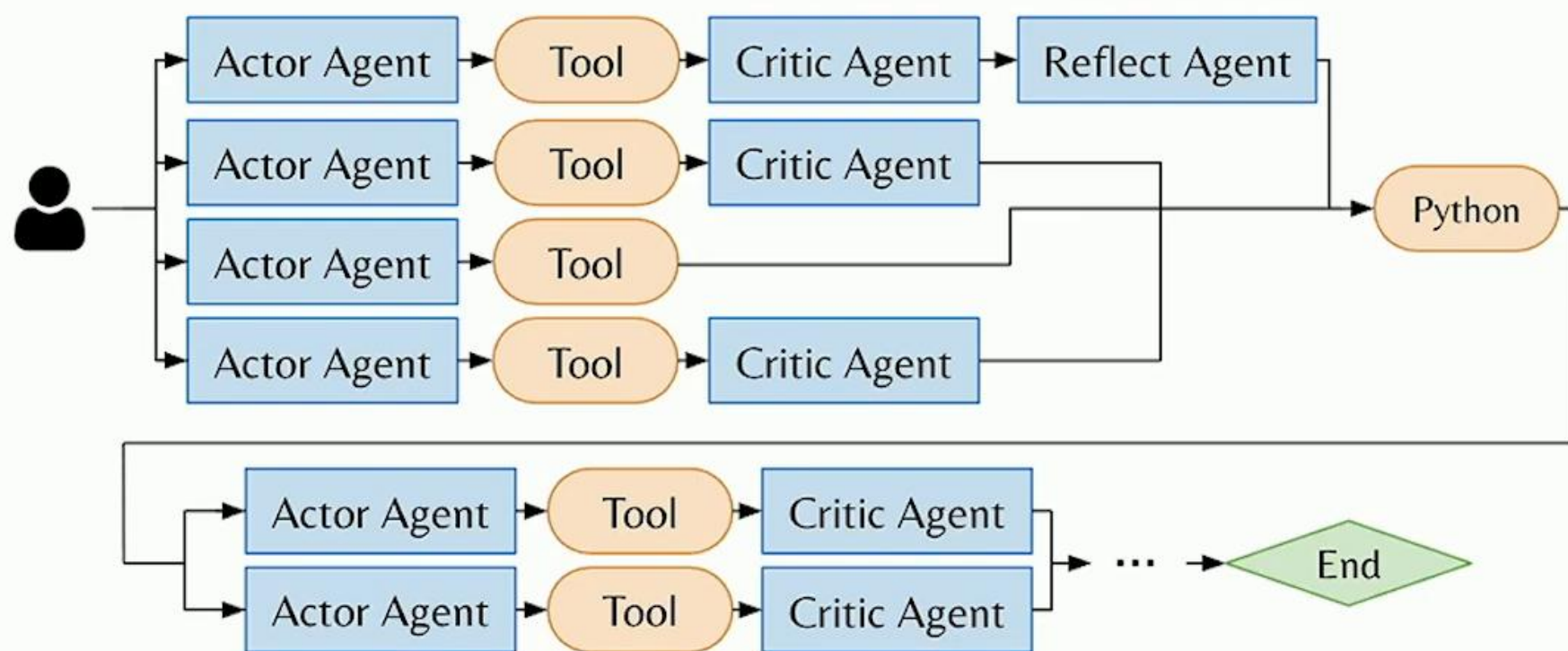
ReACT

Map-Reduce 



AI Applications are complex!

Monte Carlo Tree Search



Program Properties

1. Highly Dynamic

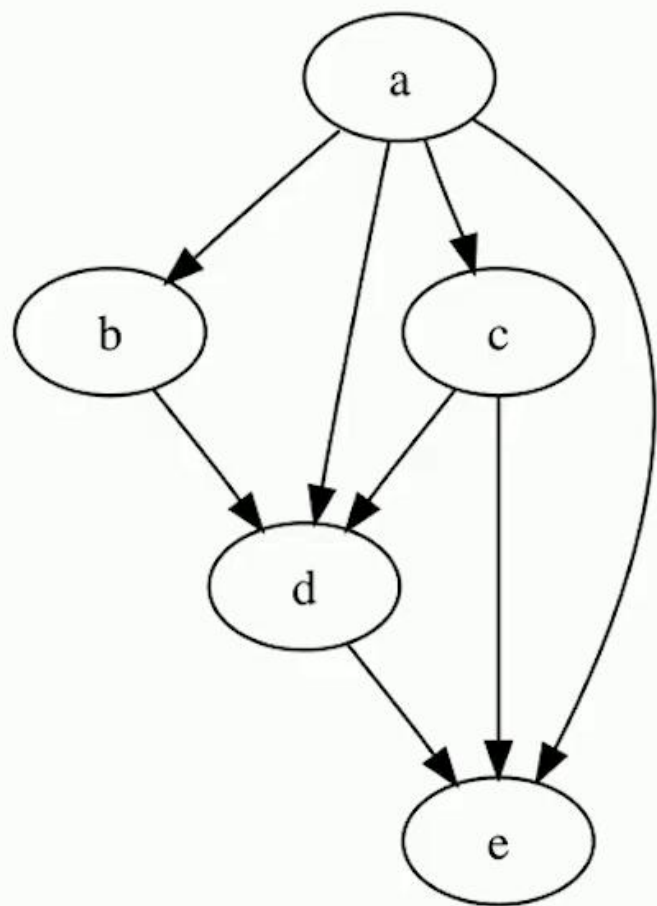
- Don't know how long program will run!

2. Distributed

- Multiple LLM calls running in parallel!

3. Non-deterministic!

- Same prompt, different choices.

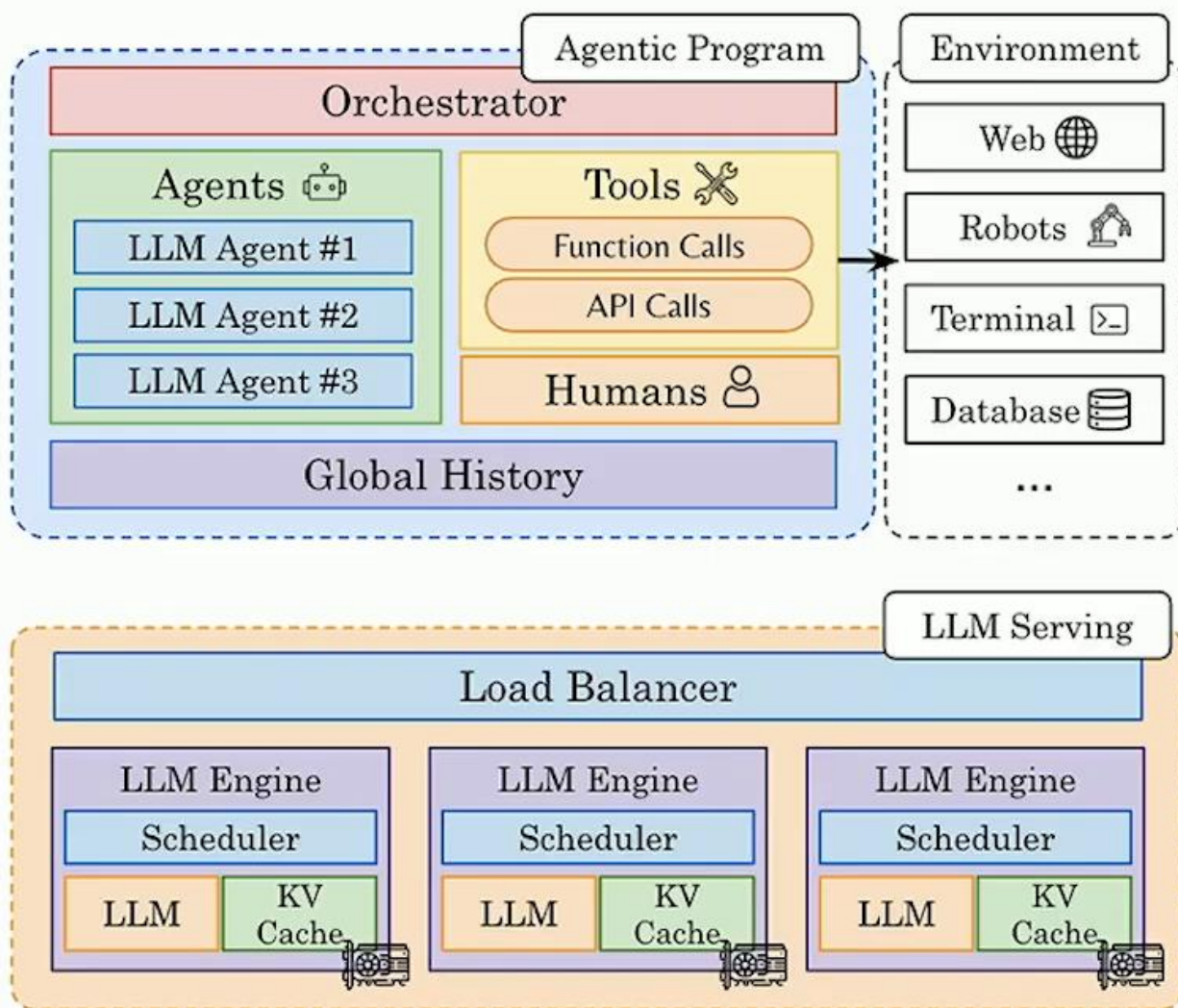


Given such **complexity** of agent programs, how to
efficiently serve them?

Agentix: Serving Engine for **Programs**, not **LLM Calls**

- **Before: Stateless** LLM serving
 - Optimize individual calls
 - TFTT, TPOT, P90/99 SLO
- **After: Stateful** LLM serving
 - Programs are *groups* of LLM calls.
 - Leverages **program-level information** *across LLM calls* to improve serving!
 - Optimize **programs**
 - Program throughput, completion-time

AI Ecosystem



Coupled!

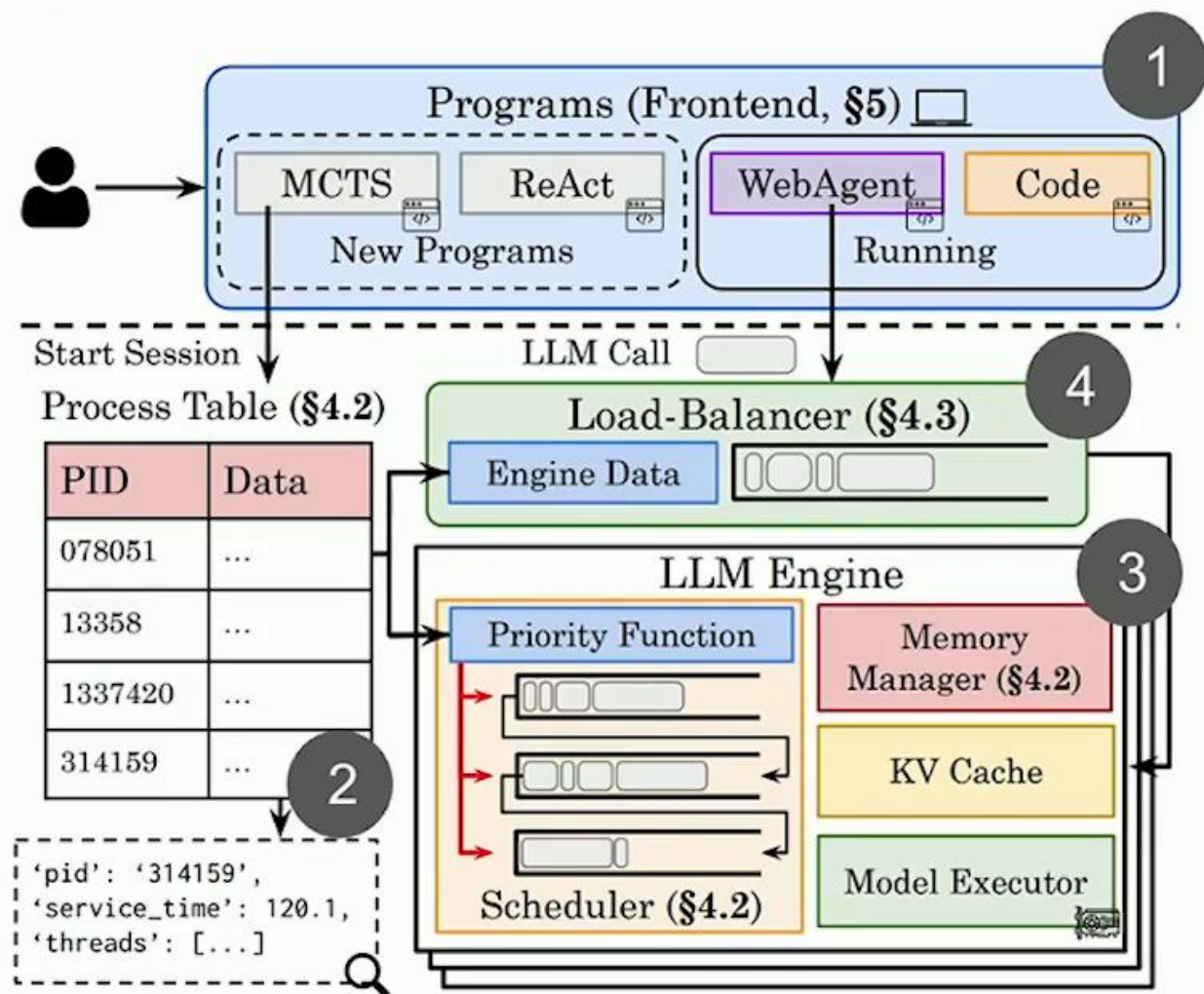
Agent Applications



LLM Serving Layer



Agentix Overview



1 Frontend - Any general Python program.

Agentix Frontend

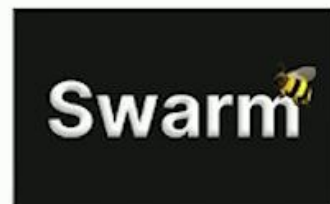
- User runs a program, which submits LLM calls over time.
 - OpenAI */v1/completions* or */v1/chat/completions*
- Frontend intercepts and annotates LLM calls with...
 - Program ID
 - Thread ID (for distributed calls)
- Enables generalization to **any program** that calls into our engine.



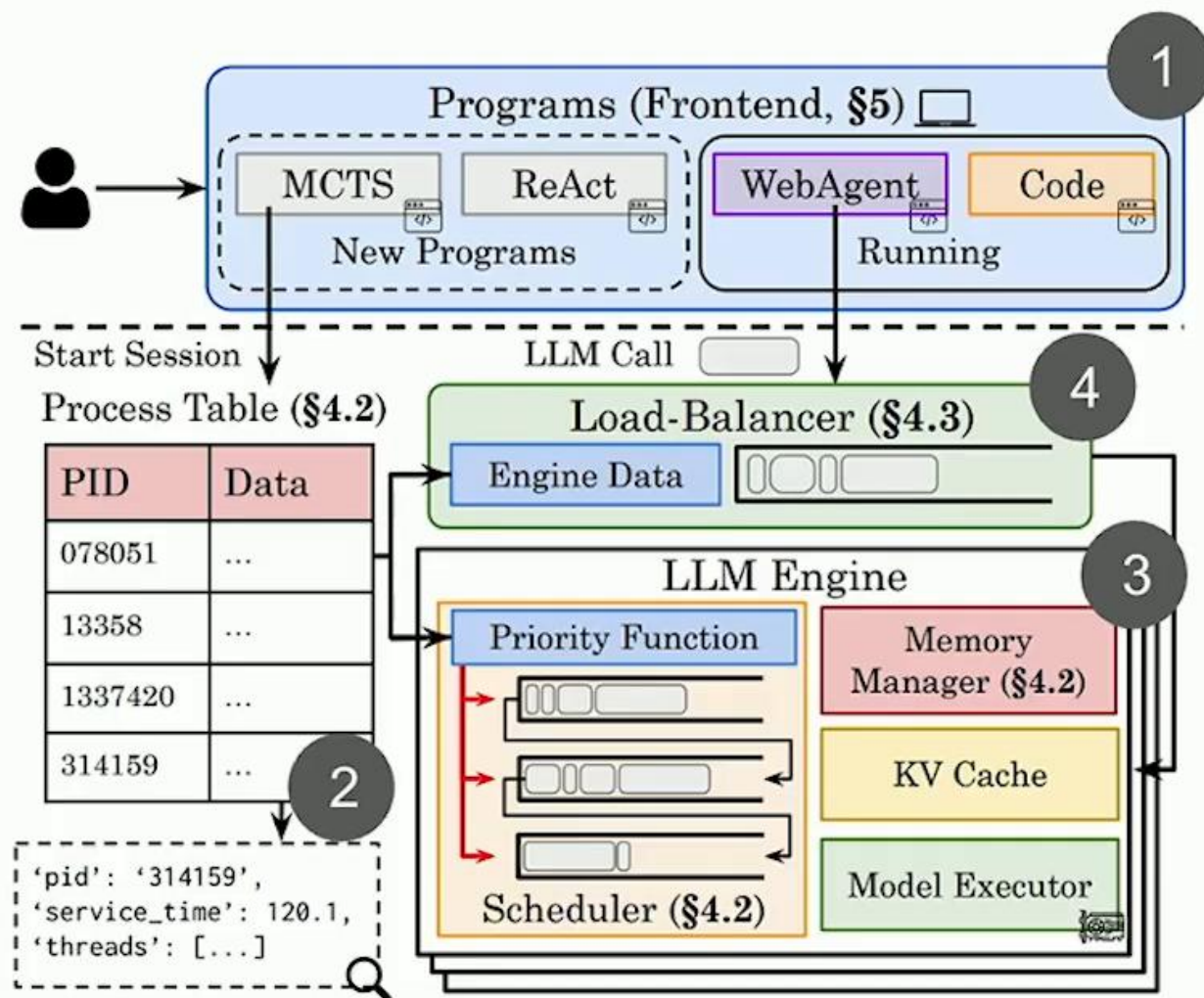
LangGraph



Microsoft
AutoGen



Agentix Overview



- 1 Frontend - Any general Python program
- 2 Process Table - Tracks running programs (PID) and data for each program.

Process Table

PID	Data
078051	...
13358	...
1337420	...
314159	...

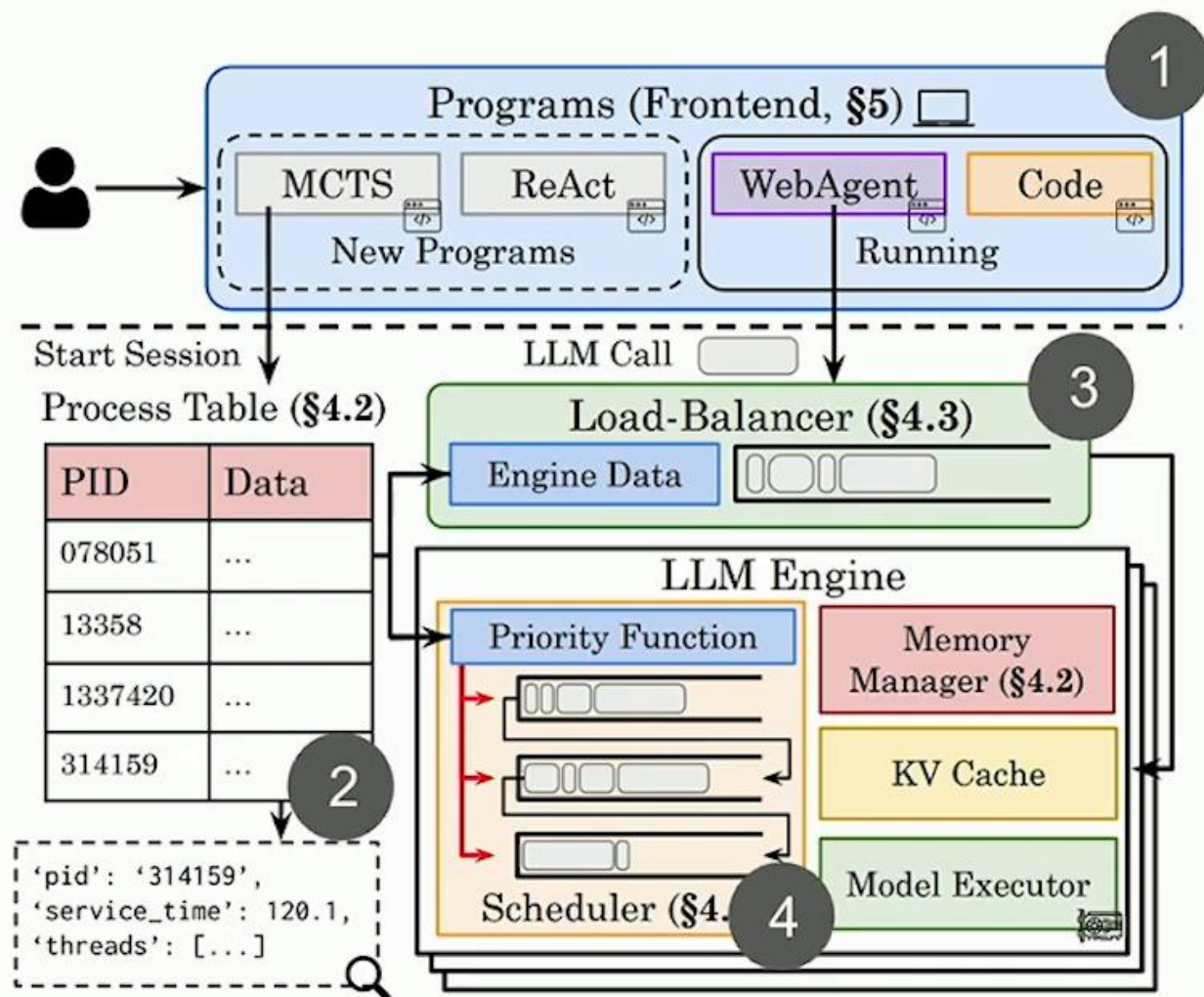
↓

```
{  
  'pid': '314159',  
  'service_time': 120.1,  
  'threads': [...]  
}
```



- API
 - start_session - Add PID to table.
 - end_session - Remove PID from table.
- Tracks **state** across LLM calls in the same program.
 - Wait/queue time on LLM engine
 - Decode/model time on engine
 - Number of parallel LLM calls
 - ...

Agentix Overview



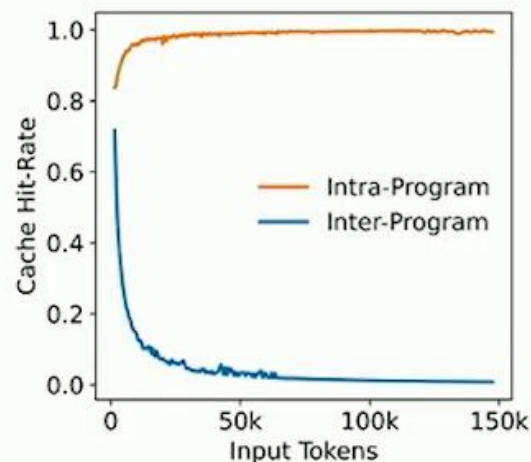
- 1 Frontend - Any general Python program
- 2 Process Table (PT) - Tracks running programs (PID) and data for each program.
- 3 Load-balancer - Leverage PT to improve data locality.

Load Balancer Analysis

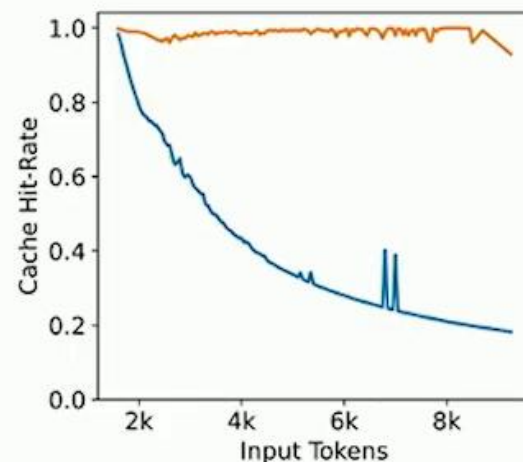
Insight #1: LLM calls in same program exhibit **high** prefix cache hit rates.

Insight #2: LLM calls in diff programs exhibit **low** prefix cache hit rates.

- Except for short requests!



(a) Single thread: Chatbot



(b) Multiple threads: MCTS

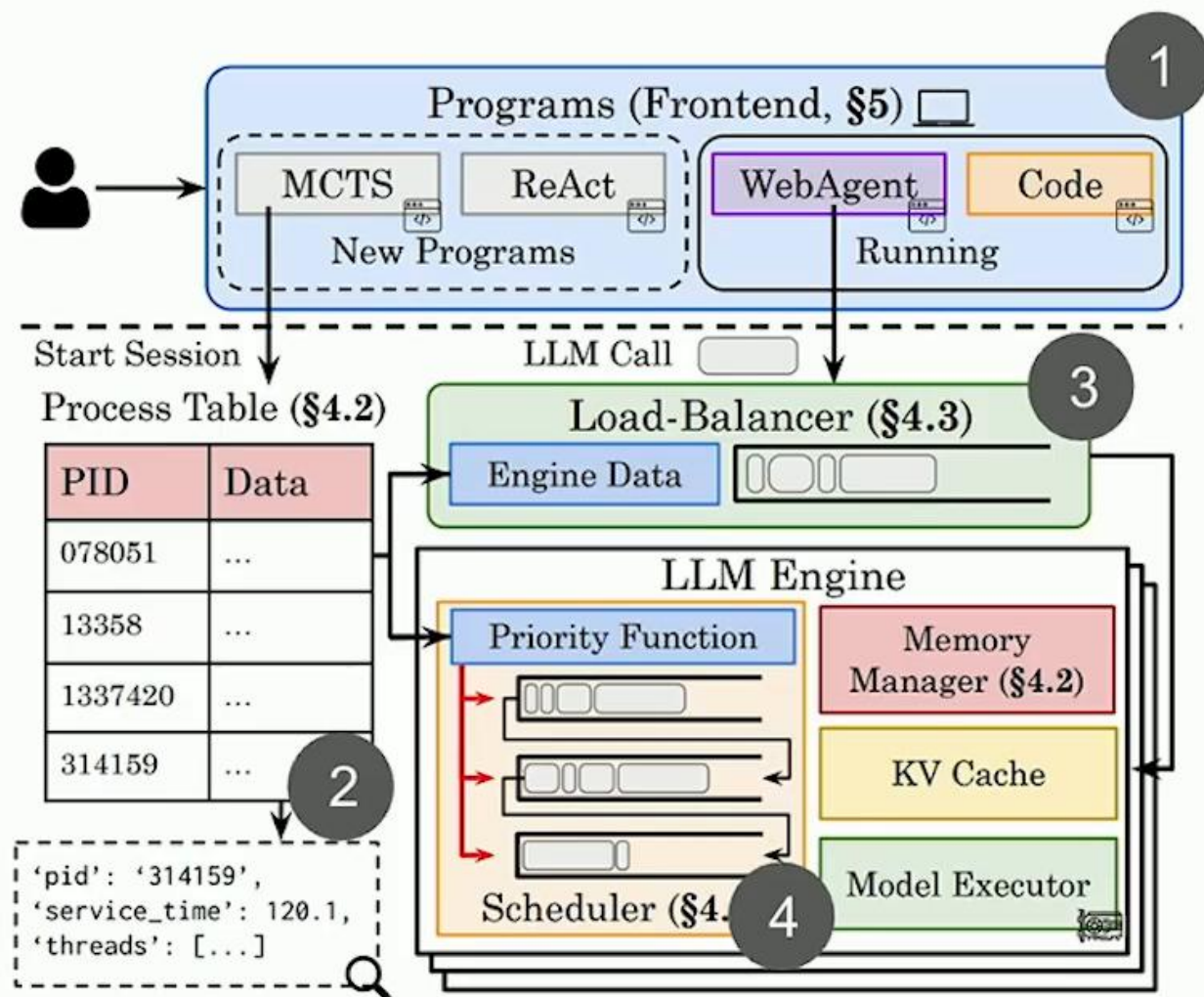
Agentix *EZ* Load-Balancer

- **Short requests** are load-balanced to *least utilized* engine.
- **Long requests** are scheduled on engine with their program PID.
 - High prefix cache hit rate!
- Same performance as **Preble**

Algorithm 2 Agentix's Load Balancer

```
1: procedure LOAD_BALANCER(Call c, Table pt, List Engines)
2:   if LEN(c.tokens) ≤ 2048 then           ▷ Small request
3:     assigned_engine = LEAST_USED(Engines)
4:   else
5:     if c.pid ∈ pt then ▷ Program already assigned to engine
6:       assigned_engine = pt[c.pid]
7:     else
8:       // Select the least utilized engine
9:       assigned_engine = LEAST_USED(Engines)
10:      pt[c.pid] = assigned_engine
11:    end if
12:  end if
13:  return assigned_engine
14: end procedure
15: procedure LEAST_USED(List Engines)
16:   // Query engine workloads in parallel
17:   workloads = QUERY_ENGINE_WORKLOADS(Engines)
18:   least_used_engine = ARGMIN(workloads)
19:   return least_used_engine
20: end procedure
```

Agentix Overview



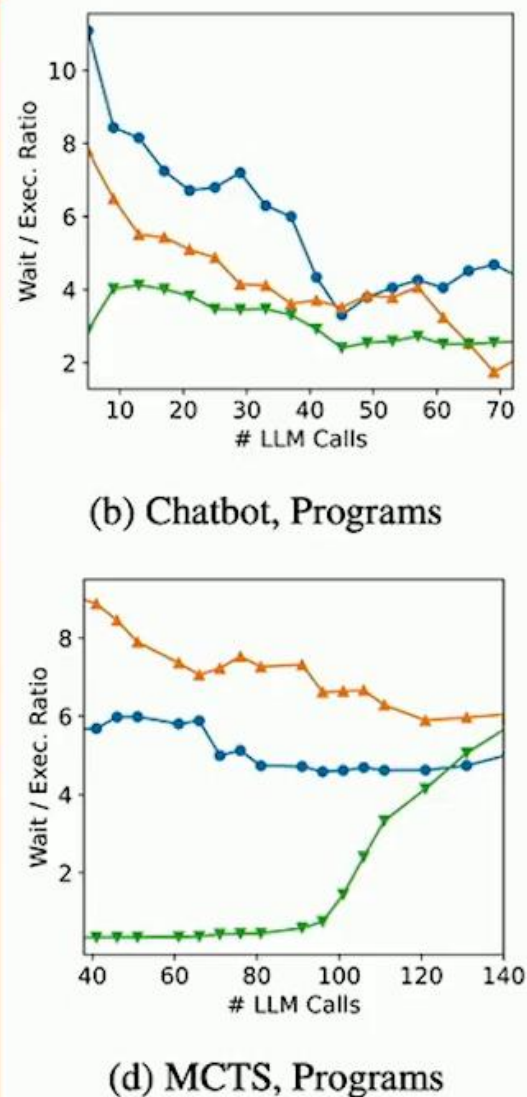
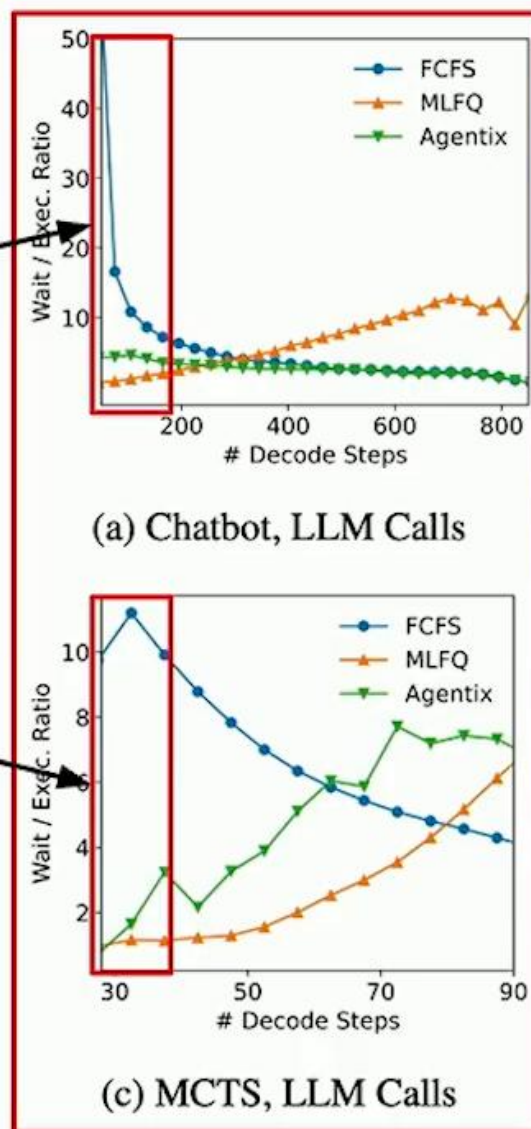
- 1 Frontend - Any general Python program
- 2 Process Table (PT) - Tracks running programs (PID) and data for each program.
- 3 Load-balancer - Leverage PT to improve data locality.
- 4 Scheduler - Leverage PT to better *prioritize* and *preempt* LLM calls based on their programs.

Problems with vLLM Scheduler

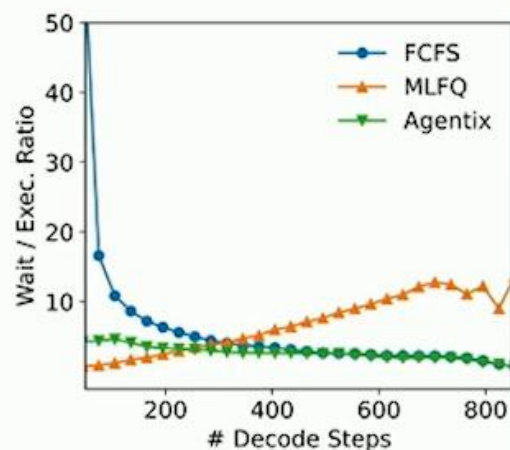
- vLLM follows First-Come-First-Serve (FCFS)
- Two problems: Head-of-line blocking at both individual **LLM calls** and **program-level**
 - **LLM call** - Long LLM calls *block* short LLM calls from completing.
 - **Program-level** - Long programs *block* short programs from completing.

vLLM Scheduler

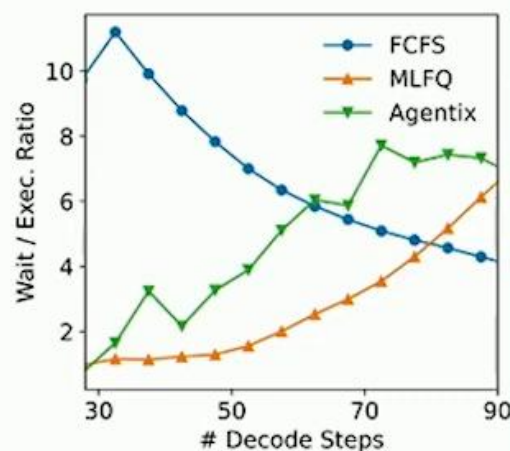
Head-of-line blocking: Short LLM requests wait excessively long.



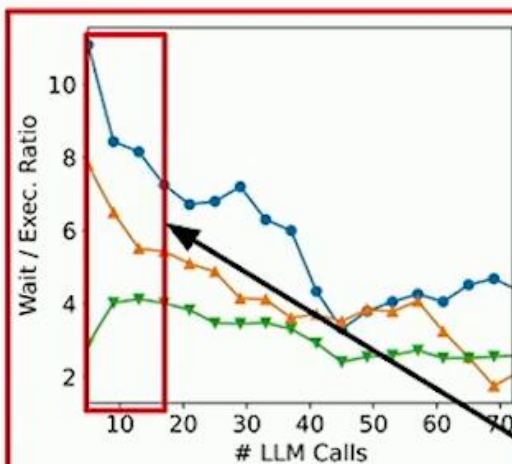
vLLM Scheduler



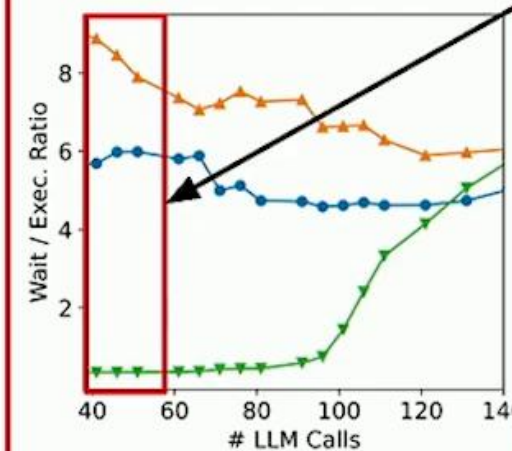
(a) Chatbot, LLM Calls



(c) MCTS, LLM Calls



(b) Chatbot, Programs



(d) MCTS, Programs

Head-of-line blocking: Short programs wait excessively long

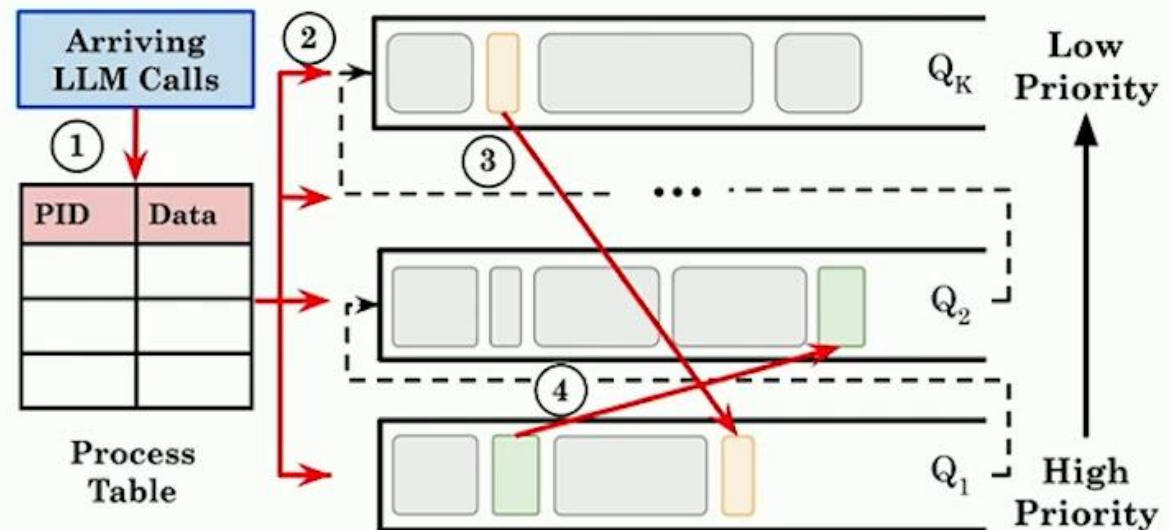
Agentix Scheduler

- **Preemption**

- Prevent different programs blocking each other.
- **Multi-level Feed Queue (MLFQ)**
 - Discretized priority leads to reduced swapping

- **Priority**

- Process Table lookup.



Agentix Scheduler Priority

- Single-threaded programs
 - Goal: Short programs should be prioritized over long programs!
 - How? No knowledge of program runtimes.
 - Can leverage cumulative runtime of a program
 - Approximate Least Attained Service (LAS) for Programs
 - First LLM calls in a program assigned highest priority...
 - Subsequent LLM calls in a program with lower priority...
 - Leverage process table to store programs' priorities.

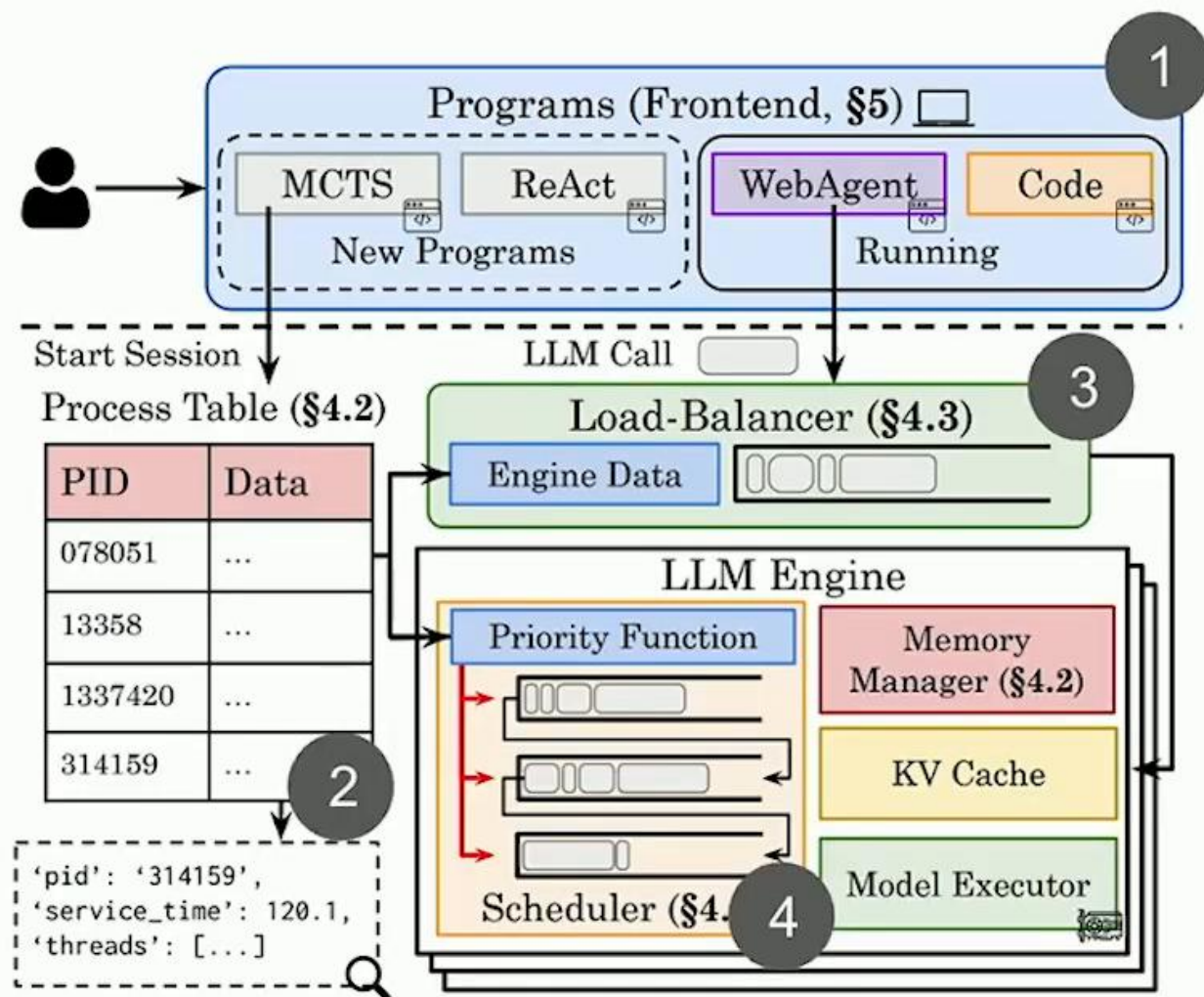
$$p(c_j) = \sum_{\substack{k < j \\ c_k.id = c_i.id}} t_k$$

Agentix Scheduler Priority

- Multi-threaded Programs
 - **Gang-schedule** - LLM calls in same program share same priority
 - Straggler mitigation :)
 - LLM calls are assigned priority based on their program's **longest critical path** thus far
 - I.e. longest running thread in a program.

$$p(c_j) = \begin{cases} 0 & \text{if } c_j \text{ is root} \\ \max_{c_k \in \mathcal{P}(c_j)} \{p(c_k) + t_k\} & \text{otherwise} \end{cases}$$

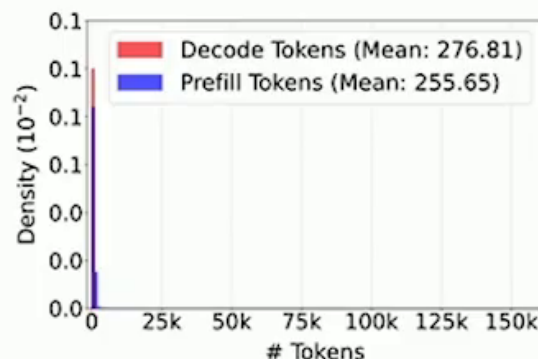
Agentix Overview



- 1 Frontend - Any general Python program
- 2 Process Table (PT) - Tracks running programs (PID) and data for each program.
- 3 Load-balancer - Leverage PT to improve data locality.
- 4 Scheduler - Leverage PT to better *prioritize* and *preempt* LLM calls based on their programs.

Evaluation

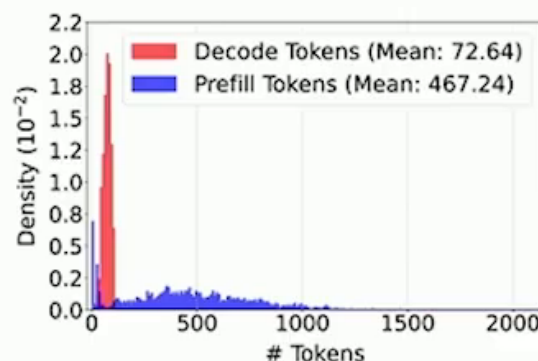
- **4 workloads** for different types of programs
 - **Chatbot:** ShareGPT
 - Entire conversation
 - Not just first turn!
 - **ReACT:** BFCL v3
 - Multitool calling
 - **MCTS/LATS:** HotpotQA
 - **Mixed**
 - $\frac{1}{3}$ sample across workloads



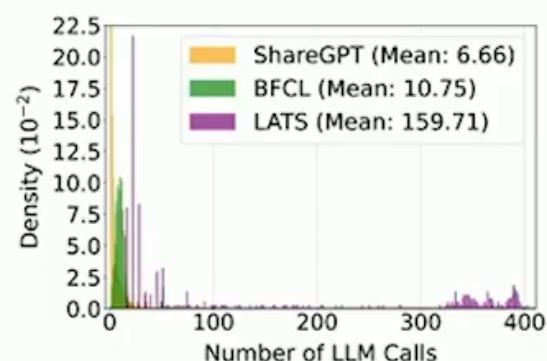
(a) ShareGPT



(b) BFCL



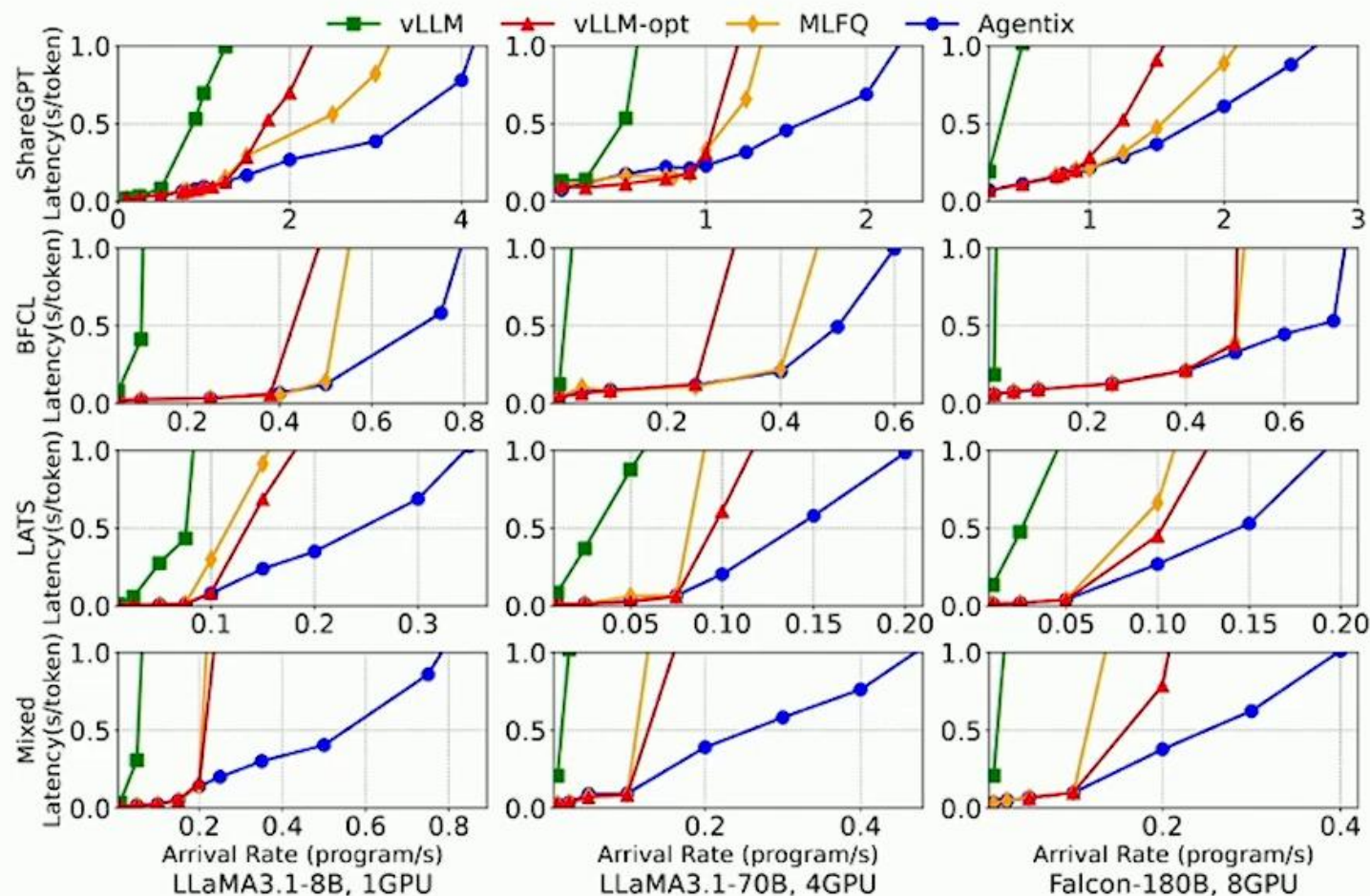
(c) LATS



(d) Number of LLM calls

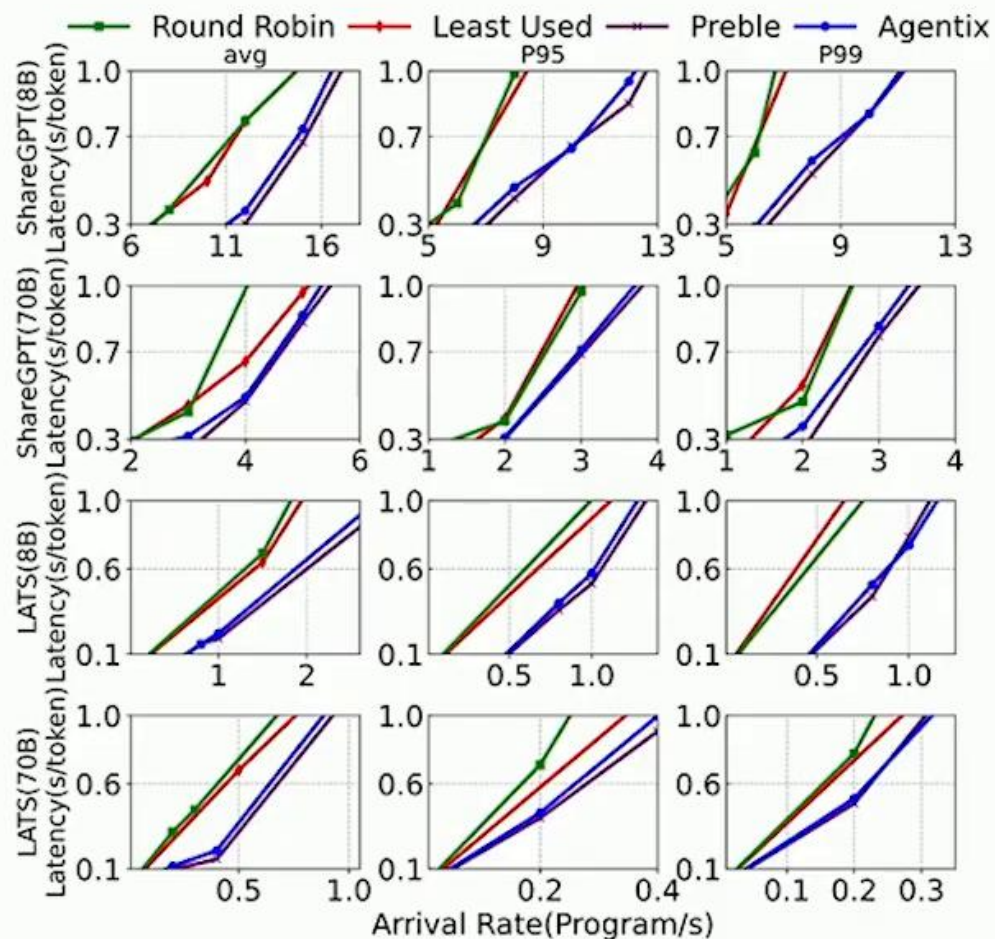
Single Engine Results

- **4-15x** higher throughput than **vLLM** given the same latency.



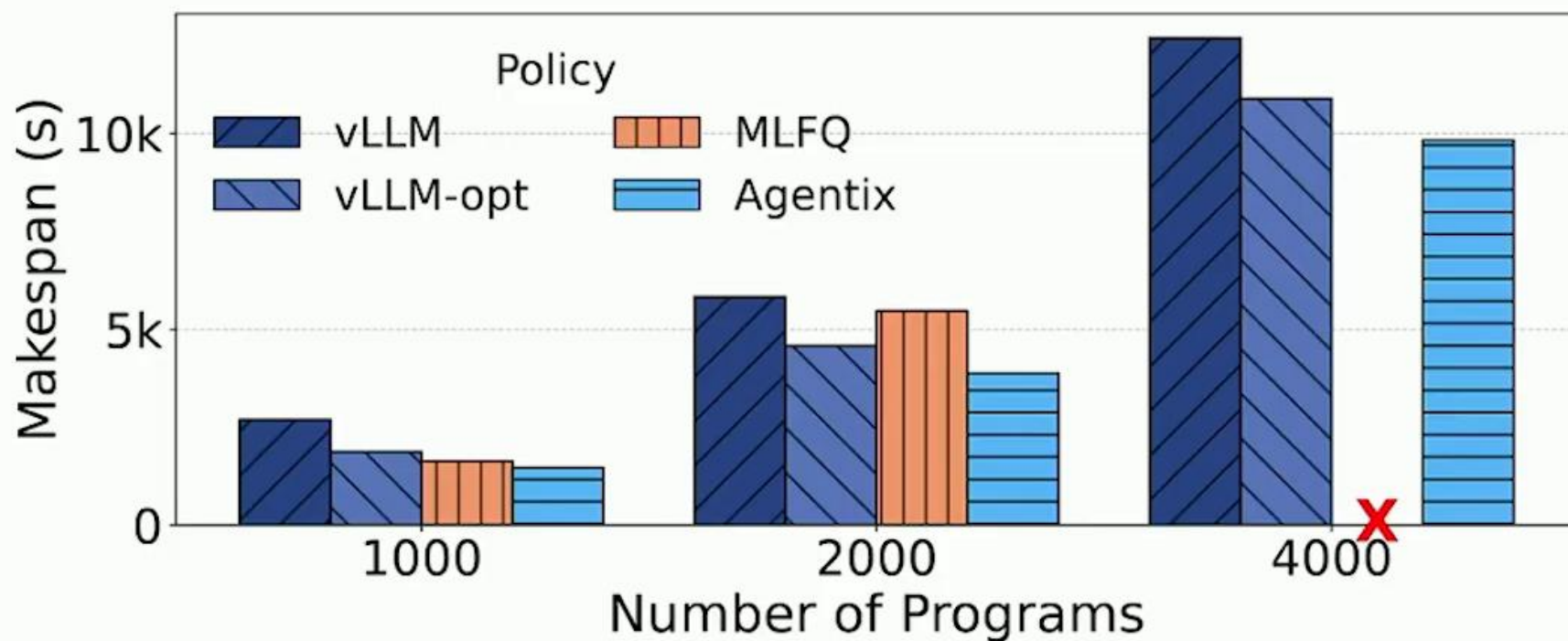
Multi Engine Results

- **1.4x** higher throughput than **standard load-balancers**
- **2x** higher throughput for tail latencies (P95/99)



Offline Inference

- Reduces makespan by up to **40%**



Conclusion



Agentix is the first serving engine for **dynamic** and **general** programs.



Agentix makes no prior assumption of programs' workloads - just works out of the box! 📦



Agentix achieves **4-15x** higher throughput than vLLM given the same latency (s/tok).

Thanks!